Applied Soft Computing 11 (2011) 3658–3670

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

A novel particle swarm optimization algorithm with adaptive inertia weight

Ahmad Nickabadi, Mohammad Mehdi Ebadzadeh*, Reza Safabakhsh

Department of Computer Engineering and Information Technology Amirkabir University of Technology 424, Hafez Ave., Tehran, 15914, Iran

ARTICLE INFO

Article history:
Received 10 March 2009
Received in revised form
27 September 2010
Accepted 30 January 2011
Available online 26 February 2011

Keywords:
Particle swarm optimization
Inertia weight
Adaptation
Success rate

ABSTRACT

Particle swarm optimization (PSO) is a stochastic population-based algorithm motivated by intelligent collective behavior of some animals. The most important advantages of the PSO are that PSO is easy to implement and there are few parameters to adjust. The inertia weight (w) is one of PSO's parameters originally proposed by Shi and Eberhart to bring about a balance between the exploration and exploitation characteristics of PSO. Since the introduction of this parameter, there have been a number of proposals of different strategies for determining the value of inertia weight during a course of run. This paper presents the first comprehensive review of the various inertia weight strategies reported in the related literature. These approaches are classified and discussed in three main groups: constant, time-varying and adaptive inertia weights. A new adaptive inertia weight approach is also proposed which uses the success rate of the swarm as its feedback parameter to ascertain the particles' situation in the search space. The empirical studies on fifteen static test problems, a dynamic function and a real world engineering problem show that the proposed particle swarm optimization model is quite effective in adapting the value of w in the dynamic and static environments.

1. Introduction

Particle swarm optimization (PSO) is a stochastic population-based algorithm which was originally introduced by Kennedy and Eberhart [1]. This algorithm is motivated by intelligent collective behavior of some animals such as flocks of birds or schools of fish. As in other evolutionary algorithms, in PSO, a population of potential solutions is evolved through successive iterations. The most important advantages of the PSO, compared to other optimization strategies, are that PSO is easy to implement and there are few parameters to adjust.

In PSO, each potential solution to an optimization problem is treated as a bird, which is also called a particle. The set of particles, also known as a swarm, is flown through the D-dimensional search space of the problem. The position of each particle is changed based on the experiences of the particle itself and those of its neighbors. The position of the ith particle is presented as

$$x_i = (x_{i1}, x_{i2}, \ldots, x_{iD})$$

where $x_{id} \in [l_d, u_d]$, $d \in [1, D]$, and l_d and u_d are the lower and upper bounds of the dth dimension of the search space. Similar to the position, the velocity of each particle is represented with a vector. The ith particle velocity is presented as $v_i = (v_{i1}, v_{i2}, \ldots, v_{iD})$. At each time step, the position and velocity of the particles are updated according to the following equations [1]:

$$v_{ij}(t+1) = v_{ij}(t) + R_{1ij}c_1(P_{ji} - x_{ij}(t)) + R_{2ij}c_2(P_{gj} - x_{ij}(t)) \quad (1)$$

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (2)$$

where R_{1ij} and R_{2ij} are two distinct random values in [0,1], c_1 and c_2 are acceleration constants, p_i is the best previous position of the particle itself and P_g denotes the best previous position of all particles of the swarm (global best PSO). If c_1 is set to 0 (and $c_2 \neq 0$), the PSO algorithm turns into the social-only model and if c_2 is set to 0 (and $c_1 \neq 0$), then it becomes a cognition-only model.

The balance between global and local search throughout the course of a run is critical to the success of an optimization algorithm [2]. Almost all of the evolutionary algorithms utilize some mechanisms to achieve this goal. The step size of the normal mutation in evolution strategies [3] and temperature parameter in simulated annealing [4] are two examples of balance controlling parameters. To bring about a balance between the exploration and exploitation characteristics of PSO, Shi and Eberhart proposed a PSO based on inertia weight in which the velocity of each particle is updated according to the following equation [5]:

$$v_{ij}(t+1) = wv_{ij}(t) + R_{1ij}c_1(P_{ij} - x_{ij}(t)) + R_{2ij}c_2(P_{gj} - x_{ij}(t)) \quad (3)$$

They claimed that a large inertia weight facilitates a global search while a small inertia weight facilitates a local search. By changing the inertia weight dynamically, the search capability is dynamically adjusted. This is a general statement about the impact of w on PSO's search behavior shared by many other researchers. However, there are situations where this rule cannot be applied

* Corresponding author. Tel.: +98 912 4598420, fax: +98 21 66413969.
E-mail address: ebadzadeh@aut.ac.ir (M.M. Ebadzadeh).

1568-4946/$ – see front matter © 2011 Elsevier B.V. All rights reserved.
doi:10.1016/j.asoc.2011.01.037

successfully. An obvious example of such situations is provided in Section 3 of this paper in which a large value of w is needed in a unimodal function optimization problem, a situation which is categorized as a local search. So, there are circumstances where a large value of w ($w \geq 08$ for example) can speed up the convergence to the optimum that PSO has already located.

Moreover, the traditional inertia weight adaptation mechanism cannot be successfully employed in some of the new PSO models proposed in the last decade. For example, in multi-swarm particle swarm optimizers the main population is divided into a number of sub-swarms with the hope of exploring different areas of the search space [6,7]. While the inertia weight is only adjusted according to the iteration number of the overall algorithm, the sub-swarms may be created at any time during a run of the algorithm and hence require a w value compatible with their needs.

In addition to the above examples, there are some optimization problems that show the need for a new inertia weight adaptation mechanism. For example, in dynamic environments with moving peaks, the global best PSO fails to follow the extrema due to the small velocity values of the particles [8]. Adaptive inertia weight strategies, as shown in the experimental section of this paper, can improve the performance of PSO in dynamic environments.

There have been several proposals to modify the PSO algorithm by utilizing an adaptive value of inertia weight in each iteration. The aim of this paper is to provide a comprehensive survey of these works, attempting to classify them based on the type of the feedback parameter used in them. To the best of our knowledge, this is the first survey on inertia weight schemes of PSO algorithms. While there are parameters that can influence the results of the PSO algorithms, this paper only focuses on inertia weight and it is not intended to cover all state of the art PSO algorithms such as CLPSO [9] and CPSO [10] (for a survey of all PSO algorithms see [11]). A new dynamic inertia weight PSO is also proposed and compared with the current methods. While the current inertia weight updating schemes use the fitness or iteration number, in the proposed method of this paper, the success rate of the swarm is used to determine the inertia weight. Experimental results show that the proposed method can properly adapt the value of the inertia weight in the static and dynamic environments.

The rest of this paper is organized as follows. Section 2 provides a review and classification of different inertia weight adaptation mechanisms. These methods are then discussed in this section. In Section 3, a new dynamic adaptive inertia weight is proposed. The set of problems used to evaluate the performance of the proposed model and the current models are provided in Section 4. Section 5 is devoted to the experimental results and Section 6 concludes the paper.

2. Inertia weight adaptation mechanisms

Since the initial development of PSO by Kennedy and Eberhart, several variants of this algorithm have been proposed by researchers. The first modification introduced in PSO was the use of an inertia weight parameter in the velocity update equation of the initial PSO resulting in Eq. (3), a PSO model which is now accepted as the global best PSO algorithm [5]. In this section, the various inertia weighting strategies are categorized into three classes. The first class contains strategies in which the value of the inertia weight is constant during the search or is determined randomly. None of these methods uses any input. In the second class, the inertia weight is defined as a function of time or iteration number and hence the strategy is referred to as time-varying inertia weight strategy. Despite claims made in some other papers, these methods are not considered adaptive since they do not monitor the situation of the particles in the search space. The third class of the inertia weight strategies contains those methods which use a feedback parameter to monitor the state of the algorithm and adjust the value of the inertia weight.

2.1. Constant and random inertia weights

Inertia weight parameter was originally introduced by Shi and Eberhart in [5]. They used a range of constant w values and showed that by using large values of w, i.e. $w > 1.2$, PSO only performs a weak exploration and with low values of this parameter, i.e. $w < 0.8$, PSO tends to traps in local optima. They suggest that with a w within the range [0.8,1.2], PSO finds the global optimum in a reasonable number of iterations. Shi and Eberhart analyzed the impact of the inertia weight and maximum velocity on the performance of the PSO in [12].

In [13], a random value of inertia weight is used to enable the PSO to track the optima in a dynamic environment.

$$\mathrm{w} = 0.5 + \frac{rand()}{2} \tag{4}$$

where $rand()$ is a random number in [0,1]; w is then a uniform random variable in the range [0.5,1]. As mentioned in Section 1, it is difficult to predict whether in a given time exploration (large values of w) or exploitation (small values of w) would be better in dynamic environments. So, a random value of w is selected to address this problem. Later in this paper, a novel more effective inertia weight strategy is proposed for both dynamic and static environments.

2.2. Time varying inertia weight strategies

Most of the PSO variants use time-varying inertia weight strategies in which the value of the inertia weight is determined based on the iteration number. These methods can be either linear or non-linear and increasing or decreasing.

In [14–16], a linear decreasing inertia weight was introduced and was shown to be effective in improving the fine-tuning characteristic of the PSO. In this method, the value of w is linearly decreased from an initial value ($w_{\max}$) to a final value ($w_{\min}$) according to the following equation:

$$w(iter) = \frac{iter_{\max} - iter}{iter_{\max}}(w_{\max} - w_{\min}) + w_{\min} \tag{5}$$

where $iter$ is the current iteration of the algorithm and $iter_{\max}$ is the maximum number of iterations the PSO is allowed to continue. This strategy is very common and most of the PSO algorithms adjust the value of inertia weight using this updating scheme.

Accepting the general idea of decreasing the inertia weight over iterations, some researchers proposed nonlinear decreasing strategies. Chatterjee and Siarry [17] propose a nonlinear decreasing variant of inertia weight in which at each iteration of the algorithm, w is determined based on the following equation:

$$w(iter) = f(iter) = \left\{ \frac{(iter_{\max} - iter)^n}{(iter_{\max})^n} \right\}(w_{\max} - w_{\min}) + w_{\min} \tag{6}$$

where n is the nonlinear modulation index. Different values of n result in different variations of inertia weights all of which start from $w_{\max}$ and end at $w_{\min}$.

Feng et al. [18,19] use a chaotic model of inertia weight in which a chaotic term is added to the linearly decreasing inertia weight. The proposed w is as follows.

$$w(iter) = (w_{\max} - w_{\min}) \times \frac{iter_{\max} - iter}{iter_{\max}} + w_{\min} \times z \tag{7}$$

where $z = 4z(1-z)$. The initial value of z is selected randomly within the range (0,1).Lei et al. use a Sugeno function as inertia weight decline curve [20]:

$$w = a = \frac{1-\beta}{1-s\beta} \tag{8}$$

where β is $iter/iter\_max$ and s is a constant larger than -1.

There are other similar approaches that use linear or nonlinear decreasing inertia weights such as [21]:

$$w = \left(\frac{2}{iter}\right)^{0.3} \tag{9}$$

Distinct from the widely used decreasing inertia weight, Zheng et al. proposed a PSO with an increasing inertia weight [22,23], and confirmed its validity in terms of convergence speed and solution precision by testing it with four standard test functions. They showed that the PSO with increasing inertia weight (increasing from 0.4 to 0.9) outperforms the PSO with decreasing inertia weight in all benchmarks used in their tests.

Another nonlinear increasing inertia weight has been proposed by Jiao et al. [24]. In this model, w is increased over time based on the following equation:

$$w(iter) = w_{\text{initial}} \times u^{iter} \tag{10}$$

where w_{initial} is the initial inertia weight value selected in the range [0,1] and u is a constant value in the range [1.0001,1.005]. In the experiments conducted in [24], u is set to 1.0002 and the performance of the algorithm is evaluated for different values of w_{initial}.

2.3. Adaptive inertia weight

The last category of inertia weight strategies studied in this paper are those that monitor the search situation and adapt the inertia weight value based on one or more feedback parameters.

In [2], a fuzzy system consisting of nine rules is proposed to dynamically adapt the inertia weight. Each rule of the system has two input and one output variables. The NCBPE (Normalized Current Best Performance Evaluation), as a performance measure of the best candidate solution found so far, and the current inertia weight value are the input variables, and the weight change is the output of the system. NCBPE is calculated as follows:

$$\text{NCBPE} = \frac{\text{CBPE} - \text{CBPE}_{\min}}{\text{CBPE}_{\max} - \text{CBPE}_{\min}} \tag{11}$$

All three fuzzy variables defined in this work have three fuzzy sets: LOW, MEDIUM and HIGH. An example of the rules of this system is:

IF (NCBPE is MEDIUM) and (weight is LOW) THEN (weight change is HIGH)

Saber et al. use the same fuzzy approach for w [25].

In [26], w is adapted based on two characteristic parameters in the search course of PSO, namely speed factor and aggregation degree factor. Speed factor is define as

$$h_i^t = \left| \frac{\min(F(pbest_i^{t-1}), F(pbest_i^t))}{\max(F(pbest_i^{t-1}), F(pbest_i^t))} \right| \tag{12}$$

where $F(pbest_i^t)$ is the fitness of the ith particle at tth iteration, and aggregation degree is defined as

$$S = \left| \frac{\min(F_{tbest}, \overline{F_t})}{\max(F_{tbest}, \overline{F_t})} \right| \tag{13}$$

where $\overline{F_t}$ is the mean fitness of all particles in the swarm at the tth iteration and F_{tbest} is the best fitness achieved by the particles at the same iteration. Using the speed and aggregation factors, the inertia weight is determined as follows:

$$w_i^t = w_{\text{initial}} - \alpha(1 - h_i^t) + \beta s \tag{14}$$

where w_{initial} is the initial value of w, and α and β are two constants typically within the range [0,1]. In [26], the performance of the PSO algorithm is evaluated for different values of α and β.There are other researchers who have used the fitness of the particles as a characteristic of the particles to adapt the inertia weight of each particle. In [27], Arumugam and Rao use the ratio of the global best fitness and the average of particles' local best fitness to determine the inertia weight in each iteration.

$$w = 1.1 - \frac{gbest}{(pbest_i)_{\text{average}}} \tag{15}$$

Table 1
Summary of various inertia weight modification mechanisms reported in the literature.

Label	Inertia weight strategy	Adaptation mechanism	Feedback parameter	Reference
W1	$w = c$	Constant w	–	[5,12]
W2	$w = 0.5 + \frac{rand()}{2}$	Random w	–	[13]
W3	$w(iter) = \frac{iter_{\max} - iter}{iter_{\max}}(w_{\max} - w_{\min}) + w_{\min}$	Linear time-varying	–	[13,15,16]
W4	$w(iter) = f(iter) = \left\{ \frac{(iter_{\max} - iter)^n}{(iter_{\max})^n} \right\}^{iter} (w_{\max} - w_{\min}) + w_{\min}$	Nonlinear time-varying	–	[17]
W5	$w(iter) = w_{\text{initial}} \times u^{iter}$	Nonlinear time-varying	–	[24]
W6	$w(iter) = (w_{\max} - w_{\min}) \frac{iter_{\max} - iter}{iter_{\max}} + w_{\min} \times z$	Linear time-varying with random changes	–	[18,19]
W7	$w(iter) = \frac{1-(iter/iter_{\max})}{1-s(iter/iter_{\max})}$	Nonlinear time-varying	–	[20]
W8	$w(iter) = \left(\frac{2}{iter}\right)^{0.3}$	Nonlinear time-varying	–	[21]
W9	$w(iter) = \frac{iter_{\max} - iter}{iter_{\max}}(w_{\min} - w_{\max}) + w_{\max}$	Linear time-varying	–	[22,23]
W10	Fuzzy rules	Adaptive	Best fitness	[2,25]
W11	$w_i^t = w_{\text{initial}} - \alpha(1 - h_i^t) + \beta s$	Adaptive	Fitness of the current and previous iterations	[26]
W12	$w = 1.1 - \frac{gbest}{(pbest_i)_{\text{average}}}$	Adaptive	Global best and average local best fitness	[27]
W13	$w_i = w_{\min} + (w_{\max} - w_{\min}) \frac{Rank_i}{Total\ \ population}$	Adaptive	Particle rank	[28]
W14	$w_{ij} = 1 - \alpha(\frac{1}{1+e^{ISA_{ij}}}), \quad ISA_{ij} = \frac{\lvert x_{ij} - P_{ij} \rvert}{\lvert p_{ij} - p_{gj} \rvert + \varepsilon}$	Adaptive	Distance to particle and global best positions	[38]
W15	$w_i = w_{\text{initial}} \left(1 \frac{dis_i}{\max\_dist}\right), \quad dist_i = \left(\sum_{d=1}^{D} (gbest_d - x_{i,d})^2 \right)^{1/2}$	Adaptive	Distance to global best position	[39]

In the adaptive particle swarm optimization algorithm proposed by Panigrahi et al., different inertia weights are assigned to different particles based on the ranks of the particles [28]:

$$w_i = w_{\min} + (w_{\max} - w_{\min})\frac{Rank_i}{Total\_population} \quad (16)$$

where $Rank_i$ is the position of the ith particle when the particles are ordered based on their particle best fitness, and *Total_population* is the number of particles. The rational of this approach is that the positions of the particles are adjusted in a way that highly fitted particles move more slowly compared to the lowly fitted ones. A summary of various existing inertia weight strategies is given in Table 1.

2.4. Discussion

Before discussing above inertia weight strategies, it is necessary to clarify the definition of some terms usually used in these strategies. Almost all of these strategies have been suggested to improve the *exploration*, *exploitation* or even both *exploration* and *exploitation* ability of the PSO. Exploitation is considered as the tendency of the optimization algorithm to refine the best solution it has found so far by searching a small vicinity of this solution. In PSO, this means that all particles converge to the same peak of the objective function and do not leave it. In contrast, the exploration characteristic of an algorithm shows the capability of the algorithm to leave the current peak and looking for better solutions.

Considering the above clarifications, we are going to investigate the impact of inertia weight on the exploration and exploitation capabilities of PSO. The inertia weight determines the contribution rate of a particle's previous velocity to its velocity at the current time step. If we ignore the impact of personal and global best position in Eq. (3) (i.e. $c_1 = c_2 = 0$), then a w value larger than 1.0 causes the particle to accelerate up to the maximum velocity and a w value less than 1.0 causes the particle to decelerate to the zero velocity [29].

When $c_1, c_2 \neq 0$, it is difficult to analyze the effect of the inertia weight. So, the general problem of function optimization with PSO is simplified to a unimodal function optimization with two particles at two different situations. At the first situation, shown in Fig. 1a, these two particles are placed far from the optimum and close to each other moving toward the optimum. The movement of the fitter particle toward the optimum is similar to the case where $c_1 = c_2 = 0$. So, large values of w make the particle move faster toward the optimum and low values of w prevent it from reaching the optimum. For the other particle, the local best position of the particle is the same as its current position. The particle velocity is then determined according to its previous velocity and its distance to the global best position. Similar to the fitter particle, larger values of w are preferred in this situation, too. It should be noted that the role of the particles may change during a course of simulation whenever the second particle reaches a better fitness compared to that of the fitter particle. The behavior of a swarm consisting of more than two particles can be easily described using the above discussion; the global-best particle always has the same situation as the more fitted particle, and each of the other particles behaves same as the less fitted particle.

In the second situation shown in Fig. 1b, the absorbing points, local and global best positions, are located near the optimum and the particle resulting in a small region that should be exploited to improve the quality of the best solution—referred to as *improvement region*. The particles, due to their high velocities near the absorbing points, move around the improvement region with a low probability of falling into it. To avoid such an oscillating movement around the improvement region, the velocity of the particle should be reduced by removing or decreasing the impact of the previous velocity of the particle. Obviously, this can be achieved by low values of inertia weight.

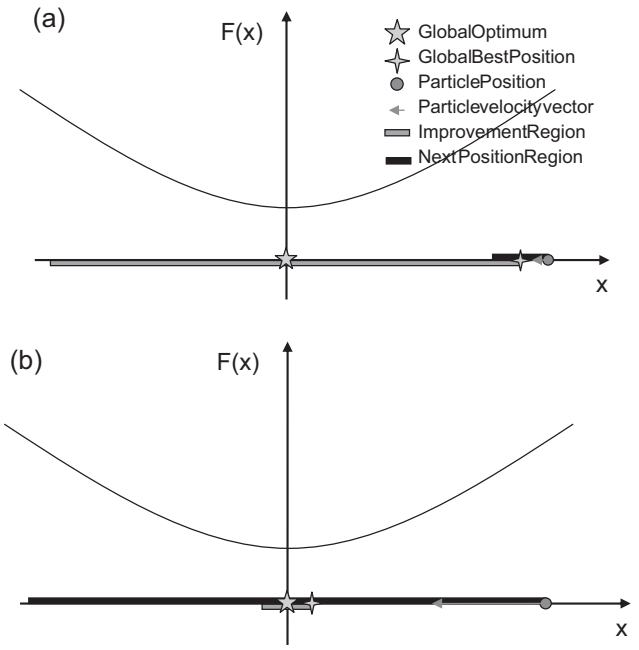

Fig. 1. Two different situations that a particle of PSO may fall in during the search. (a) Both particle's position and the global best position are far from the optimum and the particle velocity is low compared to its distance to the optimum. (b) Global best position is close to the optimum and the particle position is far from them resulting in a small improvement region and a large next position region.

During a course of simulation, particles may continually switch between two different situations of Fig. 1. This is also possible and very common to have different particles in different situations at the same iteration. The number of iterations that a particle remains in the same situation depends on the inertia weight strategy the particle uses.

```
for i=1 to number_of_particles
    Initialize x_i and v_i.
    P_i = x_i;
end
w = 1;
while(termination condition is false)
    success_count = 0;
    for i=1 to number_of_particles
        for j=1 to number_of_dimensions
            v_ij(t+1) = w * v_ij(t) + R_1ij c_1 (P_ji - x_ij(t)) + R_2ij c_2 (P_gj - x_ij(t))
            Check the velocity boundaries.
        end
        x_i = x_i + v_i;
        if(fitness(x_i) < fitness(P_i))
            success_count = success_count + 1;
            P_i = x_i;
            if(fitness(x_i) < fitness(P_g))
                P_g = x_i;
            end
        end
    end
    PS = success_count / number_of_particles;
    w = (w_max - w_min) PS + w_min
End
```

Fig. 2. Pseudo code of inertia weight adaptation in AIWPSO.

The first group of inertia weight updating schemes studied in the previous section was the random and constant inertia weight strategies. Considering the above discussion, it is clear that a constant or random inertia weight is not a proper choice. Instead, the value of the inertia weight should be determined based on the conditions of the particles.

In the second group, the value of inertia weight is either increased or decreased over time. Although the decreasing inertia weight strategies provide relatively good results, the results of the increasing inertia weight strategy of Zheng et al. [22,23] showed that there is not a strong support behind such strategies, either. Decreasing inertia weight strategies benefit from the fact that using large values of w (values in the range of [0.8,1.2]) in the first iterations increases the probability of locating the global optimum peak, and lower values of w (values in the range of [0.2,0.4]) in the rest of the run allow the particles to slowly converge to the located optimum. Depending on the decreasing scheme of the inertia weight, the global search continues few iterations and is not capable of successfully locating the global optimum in complex or high dimensional objective functions. The fine tuning phase is not optimized either. That is why the results of decreasing and increasing inertia weight strategies are almost the same. They both use a moderate value of the inertia weight to achieve a relatively good result.

In the third group, adaptive inertia weight techniques, similar to other adaptive algorithms, the environment should be monitored with some measures which are then used to adjust the parameters of the algorithm. There are two issues about this measures or feedback parameters that should be taken into account before devising an adaptive strategy. The first issue is that they should be available or easy to compute. For example, the fuzzy approach proposed by Shi and Eberhart [2] use an estimate of the optimum value of the objective function which is usually unknown, and so, it is not acceptable as a good adaptive technique. The other important factor about the feedback parameter is that it should provide good insight into the way that the inertia weight should be changed. Almost most of the proposals for adaptive inertia weight use the fitness and its derivations, such as particle's ranks, to adjust the inertia weight. Although particle's fitness value seems to be a promising feedback parameter, it should be used carefully. Referring back to our introductory discussion, it is clear that the fitness by itself is not a good criterion for evaluating the appropriateness of the inertia weight value. While the situations shown in Fig. 1a and b require, respectively, large and small values of w, both of which can occur when the fitness is high (the particles are close to the optimum) or low (the particles are far from the optimum).

It is of utmost importance that the situations that the particle is *close to* or *far from* the optimum are not determined according to distance from the optimum point. In this paper, when it is said that the particle is close to the optimum, it means that the distance of the particle from the optimum is high considering the particle's velocity. So, it would take the particle several iterations to reach the optimum with its current velocity (Fig. 1b). Similarly, the particle is close to the optimum when its distance to the optimum is low considering its velocity. So, the particle would probably oscillate around the optimum (improvement region) without or with low improvement.

3. The proposed inertia weight model

The inertia weight model proposed in this paper is inspired by the idea presented in the previous section. In Section 2.4, two different situations were introduced that a particle may fall in during a search conducted by PSO algorithms. The situation depicted in Fig. 1a needs the maximum possible w to accelerate up the movement of particles towards the optimum point while the situation depicted in Fig. 1b needs the smallest value of w to slow down the particles so that they converge to the optimum they have already located. While it is possible for all of the particles to fall in the same situation at a given iteration, the entire swarm is usually somewhere between these two points and a mild inertia weight should be chosen.

To provide an adaptive inertia weight strategy, one first needs to determine the situation of the swarm at each iteration. In this paper, the percentage of the success is used for this purpose. A high percentage of success indicates that the particles have converged to a point that is far from the optimum point and the entire swarm is slowly moving toward the optimum. Similarly, a low percentage of success shows that the particles are oscillating around the optimum without much improvement. The success rate has been utilized in some other evolutionary algorithms, too (see [3] and [30] for example).

The success of particle i at iteration t in a minimization problem is defined as:

$$S(i,t) = \begin{cases} 1 & \text{if } fit(pbest_i^t) < fit(pbest_i^{t-1}) \\ 0 & \text{if } fit(pbest_i^t) = fit(pbest_i^{t-1}) \end{cases} \quad (17)$$

where $pbest_i^t$ is the best position found by particle i until iteration t and $fit()$ is the function to be optimized. Using the success values of the particles, the success percentage of the swarm is computed as:

$$P_s(t) = \frac{\sum_{i=1}^{n} S(i,t)}{n} \quad (18)$$

where n is the number of particles and $P_s \in [0,1]$ is the percentage of the particles which have had an improvement in their fitness in the last iteration. The higher the value of P_s, the closer the situation of the swarm is to the situation of Fig. 1a, and the smaller the value of P_s, the farther the situation of the swarm is to the situation of Fig. 1b. Therefore, it is reasonable to choose the value of w proportional to P_s. This way, the inertia weight can be any increasing function of P_s define as:

$$w(t) = f(P_s(t)) \quad (19)$$

In this paper, a linear function is used to map the values of P_s to the possible range of inertia weights as follows.

$$w(t) = (w_{\max} - w_{\min})P_s(t) + w_{\min} \quad (20)$$

The range of the inertia weight ($[w_{\min}, w_{\max}]$) is selected to be [0,1.0]. Although P_s is in the range of [0,1], w can be in any acceptable range.

The main aim of the rest of this paper is to show the appropriateness of the feedback parameter P_s in delineating the situation of the particles at a given iteration during a course of run. In the next sections, some test environments are used to study the effectiveness of employing this inertia weight strategy. The overall structure of the PSO algorithm with adaptive inertia weight (AIWPSO) is shown in Fig. 2. To improve exploration characteristics of PSO, at the end of each iteration, the worst particle is replaced with a mutated version of the best particle. Mutated particle is produced by adding a Gaussian noise with zero mean standard deviation σ to a random dimension of the best particle. σ linearly decreases from the search space length to zero during a course of run.

4. Experimental setup

In order to test and compare different inertia weight strategies reviewed in this paper, a suit of commonly used static benchmark functions as well as a dynamic moving peak function are used. Static

Table 2
Test problems used in the experiments of this paper.

Function name	Test function	D	Search space	x^*	$f(x^*)$
Sphere	$f_1(x)=\sum_{i=1}^{D} x_i^2$	30	$[-100,100]^D$	[0,. . .,0]	0
Sphere	$f_2(x)=\sum_{i=1}^{D} x_i^2$	30	$[-10,190]^D$	[0,. . .,0]	0
Schwefel P2.22	$f_3(x)=\sum_{i=1}^{D} x_i+\prod_{i=1}^{D} x_i$	30	$[-10,10]^D$	[0,. . .,0]	0
Rotated hyper-ellipsoid	$f_4(x)=\sum_{i=1}^{D}\left(\sum_{j=1}^{i} x_j\right)^2$	30	$[-100,100]^D$	[0,. . .,0]	0
Noisy Quadric	$f_5(x)=\sum_{i=1}^{D} i x_i^4+rand$	30	$[-1.28,1.28]^D$	[0,. . .,0]	0
Rosenbrock	$f_6(x)=\sum_{i=1}^{D-1}\left[100(x_i^2-x_{i+1})^2+(x_i-1)^2\right]$	30	$[-5,10]^D$	[1,. . .,1]	0
Step	$f_7(x)=\sum_{i=1}^{D}\left(\lfloor x_i+0.5\rfloor\right)^2$	30	$[-10,10]^D$	[0,. . .,0]	0
Branin	$f_8(x)=(x_2-\frac{5.1}{4\pi^2}x_1^2+\frac{5}{\pi}x_1-6)^2+10(1-\frac{1}{8\pi})\cos(x_1)+10$	2	$-5\le x_1\le 10$ $0\le x_2\le 15$	$[-\pi, 12.275]$ $[\pi, 2.275]$ [9.42478, 2.475]	0.397887
Schwefel	$f_9(x)=\sum_{i=1}^{D}-x_i\sin\left(\sqrt{\lvert x_i\rvert}\right)$	30	$[-500,500]^D$	[1,. . .,1]	$-418.9829D$
Rastrigin	$f_{10}(x)=\sum_{i=1}^{D}(x_i^2-10\cos(2\pi x_i)+10)$	30	$[-5.12,5.12]^D$	[0,. . .,0]	0
Noncontinuous Rastrigin	$f_{11}(x)=\sum_{i=1}^{D} y_i^2-10\cos(2\pi y_i)+10$ $y_i=\begin{cases} x_i & \lvert x_i\rvert<0.5 \\ \frac{round(2x_i)}{2} & \lvert x_i\rvert\ge 0.5\end{cases}$	30	$[-5.12,5.12]^D$	[0,. . .,0]	0
Ackly	$f_{12}(x)=-20\exp\left(-0.2\sqrt{\frac{1}{30}\sum_{i=1}^{D} x_i^2}\right)-\exp\left(\frac{1}{D}\sum_{i=1}^{D}\cos 2\pi x_i\right)+20+e$	30	$[-32,32]^D$	[0,. . .,0]	0
Griewank	$f_{13}(x)=\frac{1}{4000}\sum_{i=1}^{D} x_i^2-\prod_{i=1}^{D}\cos(\frac{x_i}{\sqrt{i}})+1$	30	$[-600,600]^D$	[0,. . .,0]	0
Levy	$f_{14}(x)=\sin^2(\pi y_1)+\sum_{i=1}^{D-1}\left((y_i-1)^2(1+10\sin^2(\pi y_1+1))\right)^2+(y_D-1)^2(1+\sin^2(2\pi y_D))\Big\rvert$ $y_i=1+\frac{x_i-1}{4}$	30	$[-10,10]^D$	[1,. . .,1]	0
Shubert	$f_{15}(x)=\left(\sum_{i=1}^{5} i\cos((i+1)x_1+i)\right)\left(\sum_{i=1}^{5} i\cos((i+1)x_2+i)\right)$	2	$[-10,10]^D$	18 global optima	−186.7309

test functions are used to investigate the convergence speed and solution quality of the methods while the moving peak benchmark problem is used to evaluate the ability of the methods in tracking the extrema point in a dynamic environment.

4.1. Static optimization problems

To investigate the performance of the optimization algorithms, fifteen test problems are adopted in this paper. Table 2 provides a detailed description of these problems. All test functions are minimization problems. The first 8 functions ($f_1 - f_8$) are unimodal functions while the rest of the functions ($f_9 - f_{15}$) are multimodal optimization problems. The dimension of the search space (D) is 30, except for the Branin and Shubert problems which are simple two-dimensional problems. For each test problem, x^* is the best solution to the test problem and $f(x^*)$ represents the best achievable fitness for that function. All of the problems have symmetric search spaces except for f_2 in which an asymmetric search space is used to investigate the bias of the search algorithms toward the center of the search space. Among all test functions, the Noisy Quardic (f_5) is the only noisy test function.

4.2. Dynamic optimization problem

Most of the real world problems are dynamic problems in which the objective function is subject to change over time. Dynamic environments are more challenging for adaptive inertia weight strategies since after the convergence of the particles on a peak, the peak may move and the algorithm should be able to speed up the particles to move toward the new position of the peak and then slow them down to refine the result. In order to evaluate the extrema tracking performance of the proposed inertia weight model in dynamic environments, a simple form of MPB dynamic function generator [31] is used. The dynamic environment is created by moving a simple static convex function, referred to as the base function, along a trajectory. The base function used here is the parabolic equation for three dimensions:

$$f(x, y, z) = x^2 + y^2 + z^2 \tag{25}$$

At the end of some iterations, determined by the update frequency parameter, the optimum point of the base function is moved to a new point in the search space. The amount of movement is determined by the step size parameter.

5. Experimental results and discussion

In this section, the proposed strategy (AIWPSO) is compared with some other PSO variants. In the first subsection, different inertia weight adjusting strategies, including AIWPSO, are applied to some optimization problems and the results are compared based on the final accuracy and the convergence speed. The second subsection compares the results of AIWPSO with some other successful PSO algorithms. Finally, AIWPSO is applied to a real world optimization problem to show its performance in a constrained engineering problem.

In all the experiments conducted in this paper, the number of particles in the swarm is 20 and the maximum allowed number of function evaluations is 200,000, except otherwise clearly stated. The range of w ($[w_{\min}, w_{\max}]$) is [1.0,0.0] for AIWPSO and the parameters of other algorithms are set based on the recommended values in their corresponding references.

For the comparisons to be fair, all algorithms are forced to use the same number of function evaluations and the results of all experiments are averaged over 30 independent runs.

Table 3
The mean and standard deviation of the best solutions of six inertia weigh adjusting methods on 15 test problems in 200,000 function evaluations.

	AIWPSO	Decreasing	Random	Rank based	Sugeno	Increasing
f_1	3.3703E−134 (5.1722E−267)	3.0620E−38 (1.2183E−74)	6.1201E+01 (1.9840E+03)	2.6011E−46 (6.0187E−91)	3.4960E−43 (1.6756E−84)	4.0844E−39 (1.5600E−76)
f_2	1.8317E−137 (3.4534E−273)	3.8045E−40 (9.1828E−79)	5.9019E+03 (2.1256E+06)	1.4654E+02 (8.0459E+04)	1.7998E−39 (4.6322E−77)	8.2120E−26 (1.0043E−49)
f_3	1.6534E−62 (7.7348E−123)	4.7092E−25 (8.1804E−49)	1.6671E+00 (9.7657E−01)	2.2485E−28 (3.8817E−55)	2.2324E−21 (5.9247E−41)	6.4128E−24 (5.8667E−46)
f_4	1.9570E−10 (1.2012E−19)	1.9545E+00 (3.8730E+00)	7.1197E+03 (2.9550E+06)	1.3708E−01 (1.6154E−01)	2.6042E−03 (2.1459E−05)	2.3205E+00 (2.8801E+00)
f_5	5.5241E−03 (1.5358E−05)	1.3187E−02 (2.5327E−05)	1.7110E−01 (2.1552E−03)	1.4281E−02 (4.8914E−05)	8.8826E−03 (1.1683E−05)	1.2866E−02 (1.2322E−05)
f_6	2.5003E+00 (1.5978E+01)	2.8415E+01 (9.2923E+02)	5.1839E+02 (1.5639E+05)	1.9796E+01 (5.5439E+02)	2.9988E+01 (7.1726E+02)	5.8252E+01 (6.7704E+02)
f_7	0(0)	1.3333E−01(1.2381E−01)	3.2667E+00(4.4952E+00)	2.0000E−01(1.7143E−01)	0(0)	0(0)
f_8	3.9789E−01(0)	3.9789E−01(0)	3.9789E−01(0)	3.9789E−01(0)	3.9789E−01(0)	3.9789E−01(0)
f_9	−1.1732E+04(1.1409E−25)	−9.9437E+03(5.3057E+05)	−9.1904E+03(3.5170E+05)	−1.0553E+04(3.2186E+05)	−1.0906E+04(1.9079E+05)	−1.0926E+04(1.1119E+05)
f_{10}	1.6583E−01(2.1051E−01)	6.6134E+01(5.9419E+02)	1.1691E+02(1.0355E+03)	5.3330E+01(5.0639E+02)	3.7941E+01(8.4267E+01)	2.8058E+01(1.0567E+02)
f_{11}	1.1842E−16(4.2073E−31)	2.3010E+00(2.3398E+01)	1.1133E+02(7.1364E+02)	1.7067E+01(3.1121E+02)	2.3867E+01(9.0692E+01)	1.5400E+01(1.1611E+02)
f_{12}	6.9870E−15(4.2073E−31)	2.1977E+00(8.3315E+00)	3.9120E+00(1.5936E−01)	3.3299E−04(1.6507E−06)	1.4211E−14(2.1637E−29)	1.4448E−14(1.5266E−29)
f_{13}	2.8524E−02(7.6640E−04)	6.7549E−02(1.9289E−02)	1.5237E+00(1.0787E−01)	4.7321E−02(3.1969E−03)	2.1627E−02(3.9185E−04)	3.2081E−02(5.7388E−04)
f_{14}	1.4998E−32(1.2398E−94)	7.1016E+00(3.4315E+01)	1.4906E+00(8.7548E−01)	1.1456E+00(1.4902E+00)	6.0577E−02(2.5556E−02)	1.2102E−30(2.9450E−60)
f_{15}	−1.8673E+02(1.0306E−27)	−1.8673E+02(5.1930E−28)	−1.8673E+02(5.1930E−28)	−1.8673E+02(6.9239E−28)	−1.8673E+02(1.2694E−27)	−1.8673E+02(4.0390E−28)

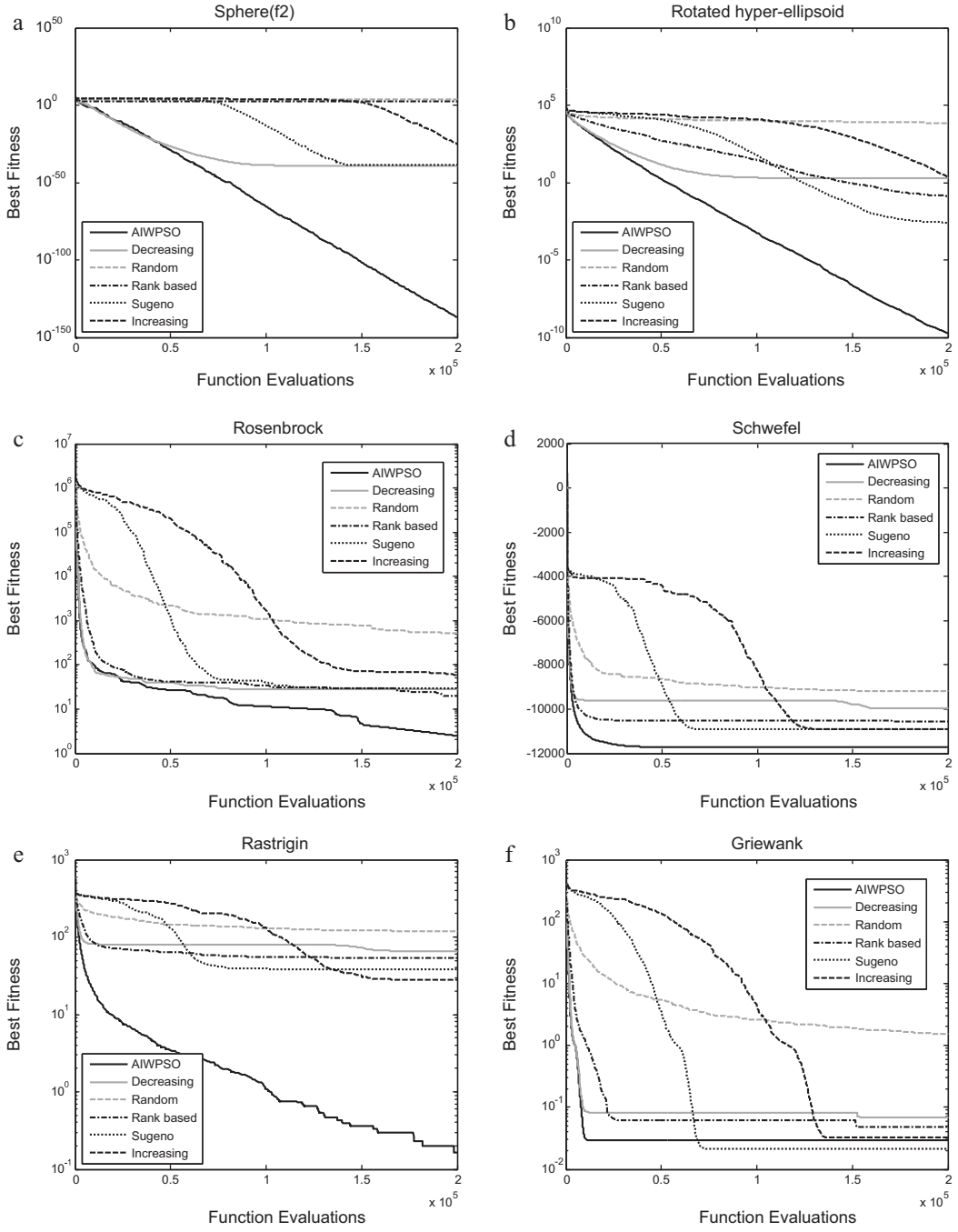

Fig. 3. The mean of the best fitness for 30 independent runs of six inertia weight adjusting methods on (a) Sphere (f_2), (b) Rotated hyper ellipsoid (f_4), (c) Rosenbrok (f_6), (d) Schwefel (f_9), (e) Rastrigin (f_{10}) and (f) Griewank (f_{13}).

5.1. Comparison with other inertia weight adjusting methods

From all strategies introduced in the previous sections, five strategies have been adopted for comparisons: decreasing inertia weight (*Linear Decreasing*) [5,12], increasing inertia weight (*Linear Increasing*) [22,23], random inertia weight (*Random*) [13], rank-based inertia weight (*Rank-based*) [28], and the Sugeno method (*Sugeno*) [20].

5.1.1. Static test functions

Five inertia weight strategies are applied to the fifteen test problems listed in Table 2 and the results are compared with those of AIWPSO. Table 3 lists the mean and standard deviation of the best solutions found by each method in 30 runs. The bold numbers indicate the best solutions for functions according to a *t*-test with significance level of 5%. The results indicate that AIWPSO provides the best accuracy in 11 out of 15 test problems (f_1–f_6, f_9–f_{11}, f_{14}), performs as well as some other algorithms in 3 test problems (f_7, f_8, f_{15}), but is outperformed only in one test function (f_{13}). Even in the case of f_{13} (Griewank test problem), AIWPSO performs close to the winner (*Sugeno*) and as Fig. 3f depicts, AIWPSO is much faster than *Sugeno* in convergence to the optimum.

A closer look at the convergence curves of different algorithms for some test functions provides more insight into their searching behavior. As Fig. 3a shows, all strategies have long periods of iterations in which the fitness is not improved much (horizontal sections of the graphs), except for AIWPSO. These are the periods when the inertia weight has not been set properly. For the Sphere and Rotated

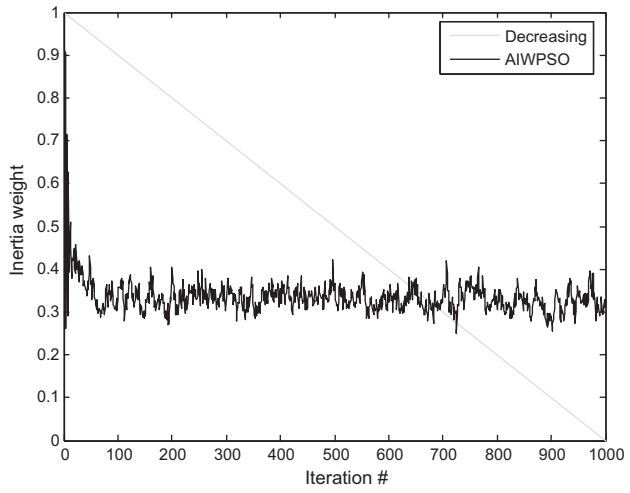

Fig. 4. Inertia weight adaptation in AIWPSO and decreasing strategy applied to Sphere test problem (f_1).

hyper ellipsoid, AIWPSO's convergence graph is a straight line in logarithmic scale. This indicates that no matter how far the best particle is from the global optimum, AIWPSO's convergence speed remains almost the same during a course of run. The convergence speed of AIWPSO is also independent from the iteration number. For the multimodal functions, the horizontal regions of the AIWPSO graphs appear due to the convergence of the algorithm to the global or a local optimum.

The performance of AIWPSO is superior due to the adaptive nature of this algorithm. Fig. 4 shows how the inertia weight is changed over time in AIWPSO applied to the Sphere test problem. The results show that the value of w is large in the first few iterations. This is mainly because of the high rate of successful movements (particle best updates) and causes the AIWPSO to perform an initial exploration in these iterations. After this short period, w converges to 0.39 and oscillates around it, a value suitable for the Sphere function.

5.1.2. Extrema tracking

In this experiment, the five PSO variants are applied to the dynamic function defined in Section 4.2. The step size is set to 4 and the position of the optimum is changed after every 100 iterations. The results are shown in Fig. 5.

The results perfectly verify the discussion about the impact of the inertia weight on the performance of PSO given in Section 2.4. As the convergence curve of the decreasing method depicts, in the initial periods (i.e. iterations less than 1000), this method cannot locate the optimum position accurately. This happens due to the high values of w during these iterations which makes the particles oscillate around the optimum. In the periods around iteration 1500, the value of w of decreasing strategy is in the range of [0.3,0.5] and this algorithm achieves its best results, though it is still slower than AIWPSO in tracking the optimum.

In contrast to the decreasing inertia weight, AIWPSO provides quite acceptable performance in all periods and never fails to locate the new position of the optimum after the environment changes. The reason for successful optimum tracking of AIWPSO is the adaptive inertia weight strategy employed in this algorithm. The way the inertia weight is adapted in AIWPSO during the search is shown in Fig. 5b. After each environment change (peak movement), the particles are placed on a point far from the optimum. This is identified using the number of successful movements of the particles which grows to a value about 1. This results in a high w which helps the particles to accelerate up toward the new optimum point. The particles take few iterations to reach this point and after that w rapidly reduces to a value around 0.4. The performance of the AIWPSO is almost the same in all periods and outperforms all other methods.

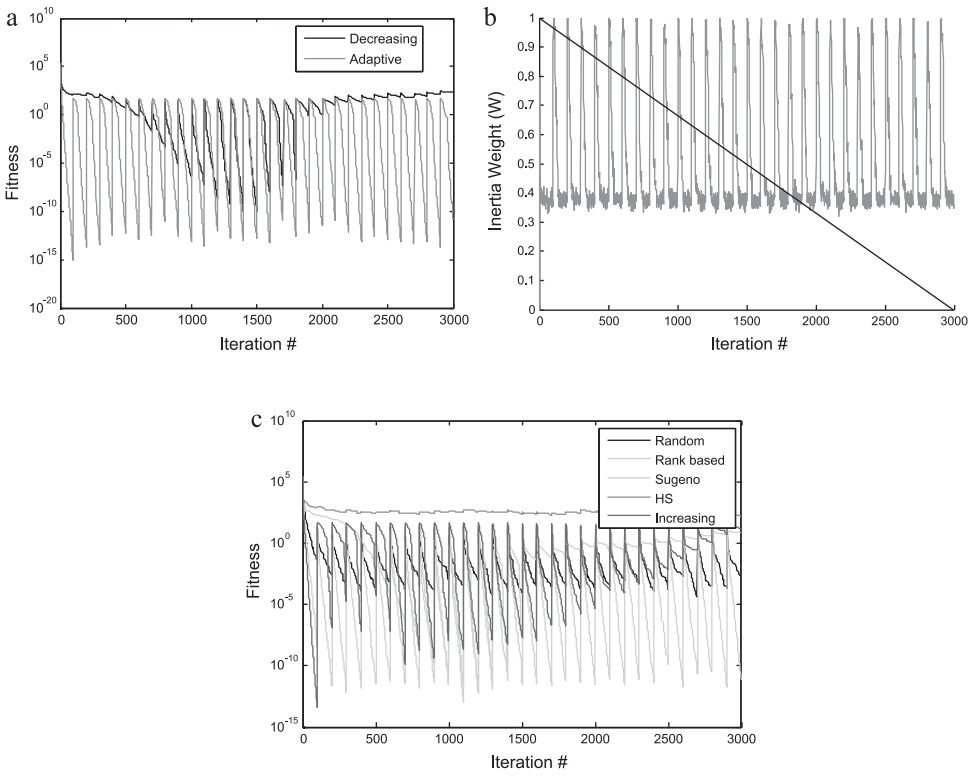

Fig. 5. Simulation results for moving peak benchmark (a) fitness of AIWPSO and Decreasing strategy, (b) inertia weight of AIWPSO and Decreasing strategy, and (c) fitness of Random, Rank based, Sugeno, HS and Increasing methods.

Table 4
The mean and standard deviation of the best solutions of six PSO variant on 15 test problems in 200,000 function evaluations.

	f_1	f_2	f_3
stdPso	5.198E−40 (1.1301E−78)	3.4621E−26(4.0873E−51)	2.0695E−25(1.4420E−49)
CPSO	5.146E−13 (7.7588E−25)	5.4282E−14(8.2868E−27)	1.2534E−07(1.1791E−14)
CLPSO	4.894E−39 (6.7814E−78)	9.9748E−39(3.7661E−84)	8.8677E−24(7.9008E−49)
FIPS	4.588E−27 (1.9577E−53)	2.6033E+02(2.1785E+04)	2.3239E−16(1.1406E−32)
Frankenstein	2.409E−16 (2.0047E−31)	5.1953E+04(1.1136E+09)	1.5804E−11(1.0301E−22)
AIWPSO	3.370E−134 (5.1722E−267)	1.8317E−137(3.4534E−273)	1.6534E−62(7.7348E−123)
	f_4	f_5	f_6
stdPso	1.4582E+00(1.1783E+00)	1.2375E−02(2.3107E−05)	2.5404E+01(5.9031E+02)
CPSO	1.8889E+03(9.9106E+06)	1.0764E−02(2.7698E−05)	8.2648E−01(2.3449E+00)
CLPSO	1.9217E+02(3.8433E+03)	4.0642E−03(9.6184E−07)	1.3217E+01(2.1480E+02)
FIPS	9.4634E+00(2.5976E+01)	3.3047E−03(8.6680E−07)	2.6714E+01(2.0025E+02)
Frankenstein	1.7315E+02(9.1577E+03)	4.1690E−03(2.4012E−06)	2.8156E+01(2.3132E+02)
AIWPSO	1.9570E−10(1.2012E−19)	5.5241E−03(1.5358E−05)	2.5003E+00(1.5978E+01)
	f_7	f_8	f_9
stdPso	0(0)	3.9789E−01(0)	−1.0995E+04(1.3753E+05)
CPSO	0(0)	3.9789E−01(1.9251E−22)	−1.2127E+04(3.3795E+04)
CLPSO	0(0)	3.9789E−01(0)	−1.2546E+04(4.2567E+03)
FIPS	0(0)	3.9789E−01(1.0520E−31)	−1.1052E+04(9.4421E+05)
Frankenstein	0(0)	3.9789E−01(1.0823E−29)	−1.1221E+04(2.7708E+05)
AIWPSO	0(0)	3.9789E−01(0)	−1.2569E+04(1.1409E−25)
	f_{10}	f_{11}	f_{12}
stdPso	3.4757E+01(1.0636E+02)	2.0956E+01(1.8327E+02)	1.4921E−14(1.8628E−29)
CPSO	3.6007E−13(1.5035E−24)	5.3717E−13(5.9437E−24)	1.6091E−07(7.8608E−14)
CLPSO	0(0)	1.3333E−01(1.1954E−01)	9.2371E−15(6.6156E−30)
FIPS	5.8502E+01(1.9185E+02)	6.1883E+01(1.4013E+02)	1.3856E−14(2.3227E−29)
Frankenstein	7.3836E+01(3.7055E+02)	7.0347E+01(2.9600E+02)	2.1792E−09(1.7187E−18)
AIWPSO	1.6583E−01(2.1051E−01)	1.1842E−16(4.2073E−31)	6.9870E−15(4.2073E−31)
	f_{13}	f_{14}	f_{15}
stdPso	2.1625E−02(4.5019E−04)	1.1422E−29(3.2335E−57)	−1.8673E+02(2.5069E−28)
CPSO	2.1245E−02(6.3144E−04)	2.0913E−15(1.2954E−29)	−1.8673E+02(6.7325E−26)
CLPSO	0(0)	1.4998E−32(1.2398E−94)	−1.8673E+02(4.1782E−28)
FIPS	2.4776E−04(1.8266E−06)	1.0273E−28(1.0052E−56)	−1.8673E+02(4.4341E−19)
Frankenstein	1.4736E−03(1.2846E−05)	5.5136E−18(1.4501E−34)	−1.8673E+02(2.9475E−16)
AIWPSO	2.8524E−02(7.6640E−04)	1.4998E−32(1.2398E−94)	−1.8673E+02(1.0306E−28)

Among the other methods, the rank-based inertia weight updating scheme is the only one capable of tracking the moving extrema. The success of the rank-based method is due the fact that in this method different inertia weight values are assigned to different particles at each iteration. Thus, there are always a number of particles with proper inertia weight values. But, as it was shown in the previous section, since the optimum inertia value is not used for all particles, its performance is less than AIWPSO. The result of the Random strategy is to some extent similar to that of the Rank-based except that the value of inertia weight is between 0.5 and 1.0 at each iteration and the algorithm cannot converge to the located optimum. The increasing method has a good performance at the first period because of its low inertia weight values and the fact that the particles are uniformly located around the optimum point. But, it fails to track the optimum in the following periods. The behavior of the Sugeno method is the same as the decreasing method and its best results are obtained when the value of inertia weight is in the range [0.2,0.5].

5.2. Comparison with other PSO variants

In this subsection, four recent successful variants of PSO are selected and their performance is compared with that of AIWPSO. The selected PSO algorithms are Cooperative PSO (CPSO) [10], Comprehensive Learning PSO (CLPSO) [9], Fully Informed Particle Swarm (FIPS) [32] and Frankenstein's PSO (FPSO) [33]. The results of the global PSO [5] are also included in the comparisons for the sake of completeness. As before, the parameters of the algorithms are set to the recommended values of the corresponding papers and all algorithms are run for the same number of function evaluations.

The above mentioned PSO-based optimization algorithms are applied to the test problems listed in Table 2. Table 4 summarizes the results in terms of mean and standard deviation of the best solutions found by different approaches in 30 runs. The best result for each test function is shown in bold. In 8 out of 15 test problems, AIWPSO performs as the best approach. In 4 test problems, it performs as well as the winner and in 3 test functions, its performance is inferior to that of the winner. There are only two test problems in which the fitness of AIWPSO is far from the best solution found by other algorithms, namely Rastrigin and Griewank. The success of CLPSO and CPSO in these two test functions is mainly due to the way they create the global best position. In both methods, the global best particle is replaced with a particle in which the value of each dimension is selected based on a tournament selection mechanism; an approach which can be included in other PSO algorithms to improve their exploration in environments similar to Rastrigin and Griewank test problems.

To have a direct and more sensible comparison between AIWPSO and other PSO variants, we have performed a *t*-test with a 5% significance level on the results of the AIWPSO and the other methods on each test function. The result of each test is that AIWPSO is significantly better, almost the same or significantly worse than the compared method on that special test function. The results of this experiment are shown in Table 5 The numbers under the name

Table 5
Comparison of AIWPSO results with those of five other PSO variants in 15 test problems based on *t*-test with 5% significance level.

AIWPSO		PSO	CPSO	CLPSO	FIPS	Frankenstein
	Significantly better	10	9	7	10	10
	Almost the same	5	6	6	2	4
	Significantly worse	0	0	2	3	1
	Overall score	10	9	5	7	9

of each method represent the number of test problems that AIWPSO is significantly better than, almost the same, or significantly worse than that method. The overall score is obtained by subtracting the third number (significantly worse) from the first number (significantly better). Once again, the results indicate the superiority of AIWPSO over other approaches. The closest competitor, CLPSO, has been outperformed by AIWPSO in seven problems and has outperformed AIWPSO only in two problems.

5.3. Real world optimization problems

In addition to the well-known test functions used in the previous experiments, the effectiveness of AIWPSO is also studied on a real world engineering problem, namely, the pressure vessel design. Here, the aim is to minimize the cost of the material, forming and welding of a cylindrical vessel. Initially introduced by Sandgren [34], the highly constrained problem of pressure vessel design can

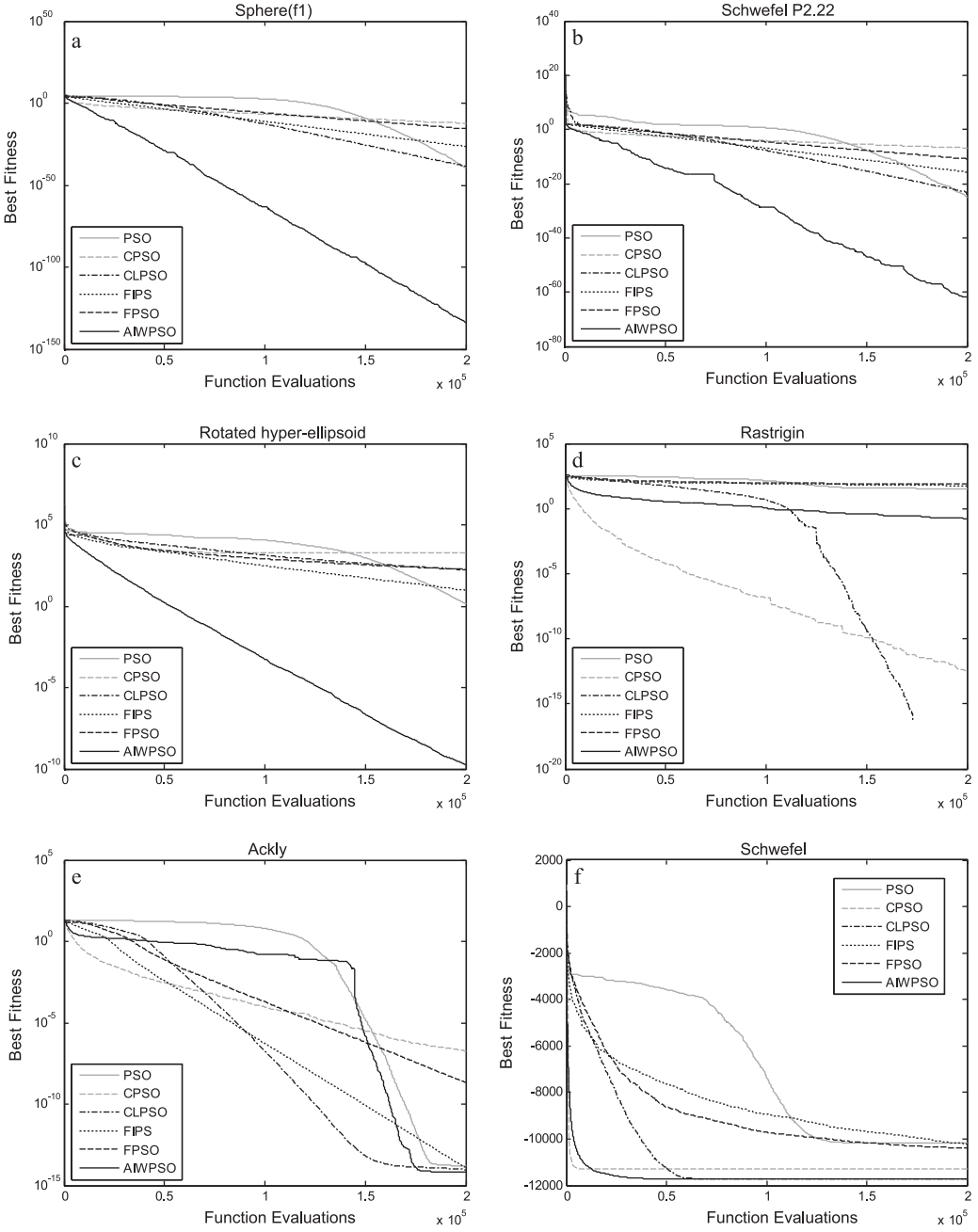

Fig. 6. . The mean of the best fitness for 30 independent runs of six PSO variant on (a) Sphere (f_1), (b) Schwefel P2.22 (f_3), (c) Rotated hyper ellipsoid (f_4), (d) Rastrigin (f_{10}), (e) Ackly (f_{12}) and (f) Schwefel (f_9).

Table 6
Optimization results for pressure vessel design problem.

	Sandgren [34]	Wu and Chow [35]	Harmony search [36]	CODEQ [37]	AIWPSO
x_1	1.125	1.125	1.125	1.125	1.125
x_2	0.625	0.625	0.625	0.625	0.625
x_3	48.97	58.1978	58.2789	58.2901554401	58.2901554404
x_4	106.72	44.293	43.7549	43.6926562409	43.6926562409
$f(x)$	7980.894	7207.494	7198.433	7197.728	7197.728

be formulated as follows.

$$\text{Minimize } f(x) = 0.6224x_1x_3x_4 + 1.7781x_2x_3^2 + 3.1611x_1^2x_4 + 19.84x_1^2x_4 + 19.84x_1^2x_3$$

$$\text{Subject to: } g_1(x) = 0.0163x_3 - x_1 \le 0,$$
$$g_2(x) = 0.00954x_3 - x_2 \le 0,$$
$$g_3(x) = 1296000 - \pi x_3^2x_4 - \frac{4}{3}\pi x_3^3 \le 0,$$
$$g_4(x) = x_4 - 240 \le 0,$$
$$g_5(x) = 1.1 - x_1 \le 0,$$
$$g_6(x) = 0.6 - x_2 \le 0,$$

The four design parameters are shell thickness (x_1), spherical head thickness (x_2), radius of cylindrical shell (x_3) and shell length (x_4). x_1 and x_2 are integer multipliers of 0.0625. x_3 and x_4 are continuous variables in the ranges of $40 \le x_3 \le 80$ and $20 \le x_4 \le 60$.

Table 6 summarizes the optimization results of AIWPSO, Sandgren [34], genetic algorithm of Wu and Chow [35], Harmony search [36], and CODEQ [37] on the pressure vessel design problem. For each approach, the best found solution and the cost of the corresponding solution are reported. The reported results of AIWPSO are the minimum result of 10 independent runs each of which has continued for 50,000 fitness evaluations. The results of the other approaches are taken from [37]. Among the five tested algorithms, AIWPSO and CODEQ provide the best results.

6. Conclusions

In this paper, different inertia weight models suggested to improve the performance of PSO are reviewed. These methods are categorized in three groups: constant, time varying and adaptive inertia weight strategies. The first group which cannot be considered adaptive uses constant or random inertia weights. In the second group, w is a function of iteration, but the state of the environment or search algorithm is not checked during a course of run. These methods cannot be accepted as adaptive inertia weight models, either. Even in the last group of inertia weight strategies in which some feedback parameters are used, the experimental results shown in Fig. 3 and the discussion of Section 2.4 show that the parameters are not properly adjusted. From the current methods, the rank-based inertia weight method proposed in [28] is the only updating scheme that can provide good results in both static and dynamic environments. Fig. 6

This paper proposed a new adaptive inertia weight strategy based on the success rate of the particles. In this strategy the success rate of the particles is used as a feedback parameter to realize the state of the particles in the search space and hence to adjust the value of inertia weight. Experimental results of Section 5 clearly prove the superiority of the proposed model over other inertia weight adjusting models as well as other successful PSO variants. The comparisons are made in terms of convergence speed and solution accuracy.

In this paper, the appropriateness of the proposed feedback parameter was evaluated and there was no attempt to find the optimum map from the success rate of the particles to the inertia weight values. This work can be done using the genetic programming (GP) algorithm. It is also possible to use the success rate of each particle separately and employ it to assign different inertia weight values to different particles at the same iteration.

References

[1] J. Kennedy, R.C. Eberhart, Particle swarm optimization, in: IEEE International Conference on Neuran Networks, 1995, pp. 1942–1948.
[2] Y. Shi, R. Eberhart, Fuzzy adaptive particle swarm optimization, in: Congress on Evolutionary Computation Seoul, Korea, 2001.
[3] I. Rechenberg, Evolutions strategie: Optimierung technischer Systeme nach Prinzipien der biologischen Evolution, Frommann-Holzboog, Stuttgart, 1973.
[4] S. Kirkpatrick, C.D. Gelatt, M.P. Vecchi, Optimization by simulated annealing, Science 220 (1983) 671–680.
[5] Y.H. Shi, R.C. Eberhart, A modified particle swarm optimizer, in: IEEE International Conference on Evolutionary Computation, Anchorage Alaska, 1998, pp. 69–73.
[6] X. Li, Adaptively choosing neighborhood bests using species in a particle swarm optimizer for multimodal function optimization, in: GECCO 2004, Seattle, USA, 2004, pp. 105–116.
[7] A. Nickabadi, M.M. Ebadzadeh, R. Safabakhsh, DNPSO: a dynamic niching particle swarm optimizer for multi-modal optimization, in: IEEE World Congress on Computational Intelligence (WCCI), 2008.
[8] A. Nickabadi, M.M. Ebadzadeh, R. Safabakhsh, Evaluating the performance of DNPSO in dynamic environments, in: IEEE International Conference on Systems, Man, and Cybernetics, Singapore, 2008.
[9] J.J. Liang, A.K. Qin, P.N. Suganthan, S. Baskar, Comprehensive learning particle swarm optimizer for global optimization of multimodal functions, in: IEEE T. on Evolutionary Computation, vol. 10, 2006, pp. 281–295.
[10] F.v.d. Bergh, A.P. Engelbrecht, A cooperative approach to particle swarm optimization, in: IEEE Transaction on Evolutionary Computation, vol. 8, 2004, pp. 225–239.
[11] R. Poli, J. Kennedy, T. Blackwell, Particle swarm optimization, an overview, Swarm Intelligence 1 (2007) 33–57.
[12] Y.H. Shi, R.C. Eberhart, Parameter selection in particle swarm optimization, in: Annual Conference on Evolutionary Programming, San Diego, 1998.
[13] R.C. Eberhart, Y.H. Shi, Tracking and optimizing dynamic systems with particle swarms, in: Congress on Evolutionary Computation, Korea, 2001.
[14] R.C. Eberhart, Y.H. Shi, Comparing inertia weights and constriction factors in particle swarm optimization, in: IEEE Congress on Evolutionary Computation, 2000, pp. 84–88.
[15] Y.H. Shi, R.C. Eberhart, Empirical study of particle swarm optimization, in: Congress on Evolutionary Computation, Washington DC, USA, 1999.
[16] Y.H. Shi, R.C. Eberhart, Experimental study of particle swarm optimization, in: SCI2000 Conference, Orlando, 2000.
[17] A. Chatterjee, P. Siarry, Nonlinear inertia weight variation for dynamic adaption in particle swarm optimization, Computer and Operations Research 33 (2006) 859–871, March 2006.
[18] Y. Feng, G. Teng, A. Wang, Y.M. Yao, Chaotic inertia weight in particle swarm optimization, in: Second International Conference on Innovative Computing, Information and Control (ICICIC 07), 2007, pp. 475–1475.
[19] Y. Feng, Y.M. Yao, A. Wang, Comparing with chaotic inertia weights in particle swarm optimization, in: International Conference on Machine Learning and Cybernetics, August 2007, 2007, pp. 329–333.
[20] K. Lei, Y. Qiu, Y. He, A new adaptive well-chosen inertia weight strategy to automatically harmonize global and local search ability in particle swarm optimization, in: ISSCAA, 2006.
[21] S. Fan, Y. Chiu, A decreasing inertia weight particle swarm optimizer, Engineering Optimization 39 (2007) 203–228.
[22] Y. Zheng, L. Ma, L. Zhang, J. Qian, Empirical study of particle swarm optimizer with an increasing inertia weight, in: IEEE Congress on Evolutionary Computation, 2003.
[23] Y. Zheng, L. Ma, L. Zhang, J. Qian, On the convergence analysis and parameter selection in particle swarm optimization, in: Proceedings of the Second International Conference on Machine Learning and Cybernetics, 2003.
[24] B. Jiao, Z. Lian, X. Gu, A dynamic inertia weight particle swarm optimization algorithm, Chaos, Solitons & Fractals 37 (2008) 698–705.
[25] A.Y. Saber, T. Senjyu, N. Urasaki, T. Funabashi, Okinawa unit commitment computation—a novel fuzzy adaptive particle swarm optimization approach, in: Power Systems Conference and Exposition, 2006, pp. 1820–1828.
[26] X. Yang, J. Yuan, H. Mao, A modified particle swarm optimizer with dynamic adaptation", Applied Mathematics and Computation 189 (2007) 1205–1213.
[27] M.S. Arumugam, M.V.C. Rao, On the improved performances of the particle swarm optimization algorithms with adaptive parameters, cross-over opera-

tors and root mean square (RMS) variants for computing optimal control of a class of hybrid systems, Applied Soft Computing (2008) 324–336.
[28] B.K. Panigrahi, V.R. Pandi, S. Das, Adaptive particle swarm optimization approach for static and dynamic economic load dispatch, Energy Conversion and Management 49 (2008) 1407–1415.
[29] F.V.D. Bergh, An analysis of particle swarm optimizers, PhD, University of Pretoria, 2001.
[30] H.G. Beyer, H.P. Schwefel, Evolution Strategies: A Comprehensive Introduction Natural Computing, vol. 1, 2002, pp. 3–52.
[31] J. Branke, Memory enhanced evolutionary algorithms for changing optimization problems, Congress on Evolutionary Computation (1999) 1875–1882.
[32] R. Mendes, J. Kennedy, J. Neves, The fully informed particle swarm: simpler, maybe better, IEEE Transactions on Evolutionary Computation 8 (2004) 204–210.
[33] M.A.M.d. Oca, T. Stutzle, M. Birattari, M. Dorigo, Frankenstein's pso: a composite particle swarm optimization algorithm, IEEE Transactions on Evolutionary Computation 13 (2009) 1120–1132.
[34] E. Sandgren, Nonlinear integer and discrete programming in mechanical design optimization, ASME 112 (1990) 223–229.
[35] S.J. Wu, P.T. Chow, Genetic algorithms for nonlinear mixed discrete-integer optimization problems via metagenetic parameter optimization, Engineering Optimization 24 (1995) 137–159.
[36] K. Lee, Z. Geem, A new meta-heuristic algorithm for continuous engineering optimization: harmony search theory and practice, Computer Methods in Applied Mechanics and Engineering 194 (2004) 3902–3933.
[37] M.G.H. Omran, CODEQ: an effective metaheuristic for continuous global optimisation, International Journal of Metaheuristics 1 (2010) 108–131.
[38] Z. Qin, F. Yu, Z. Shi, Y. Wang, Adaptive inertia weight particle swarm optimization, in: ICAISC 2006, 2006, pp. 450–459.
[39] K. Suresh, S. Ghosh, D. Kundu, A. Sen, S. Das, A. Abraham, Inertia-adaptive particle swarm optimizer for improved global search, in: Eighth International Conference on Intelligent Systems Design and Applications—ISDA 2008, 2008.